

Da dati bibliografici eterogenei a uno schema unificato: pipeline ETL per Bibliometrix-Python

Panoramica del progetto

Questa Pull Request introduce una pipeline ETL in Python per Bibliometrix-Python, con l'obiettivo di rendere la dashboard piu' robusta nella gestione di dati bibliografici provenienti da sorgenti eterogenee.

L'implementazione Python originale funziona in modo affidabile soprattutto con dati simili a Web of Science. Tuttavia, esportazioni bibliografiche provenienti da sorgenti come Scopus, PubMed, Dimensions, Lens, Cochrane, OpenAlex e Web of Science utilizzano nomi di campo, formati, delimitatori e tipi di dato differenti. Di conseguenza, alcune funzioni analitiche possono fallire quando il DataFrame in input non rispetta esattamente lo schema interno atteso.

Questa PR implementa un livello ETL source-agnostic ispirato al ruolo concettuale di `convert2df()` nel pacchetto R `bibliometrix`. L'obiettivo e' estrarre record grezzi, trasformarli in uno schema unificato simile a Web of Science, validare i tipi risultanti e caricare il DataFrame standardizzato nella dashboard Shiny esistente.

Problemi affrontati

L'implementazione precedente presentava diverse limitazioni:

- assenza di un punto di ingresso centralizzato equivalente a `convert2df()`;
- logica di trasformazione distribuita in piu' punti;
- dipendenza implicita da colonne e formati simili a Web of Science;
- gestione non sempre robusta di valori mancanti come `NaN` e `None`;
- campi multi-valore non sempre garantiti come liste Python;
- alcune funzioni analitiche assumevano direttamente un oggetto reattivo Shiny e chiamavano `.get()`;
- alcune funzioni non validavano colonne richieste e tipi prima del calcolo;
- analisi basate sulle citazioni potevano produrre errori con citazioni mancanti o pari a zero.

La pipeline ETL introdotta risolve questi problemi imponendo uno schema target stabile prima che i dati raggiungano il livello analitico.

Come la soluzione risponde alle richieste della traccia

La traccia richiede una pipeline ETL robusta, non una singola funzione monolitica. Questo requisito viene risolto separando il lavoro in moduli distinti: `file_extractor.py` si occupa solo dell'estrazione dei file, `api_retriever.py` gestisce il recupero via API, `standardizer.py` contiene la trasformazione verso lo schema comune, mentre `validation.py` controlla che il risultato finale rispetti il contratto previsto. In questo modo ogni fase ha una responsabilita' chiara e la pipeline puo' essere controllata o estesa senza modificare tutto il sistema.

All'interno della fase Extract rientrano anche i parser in `parsers.py`. Sono stati aggiunti per gestire formati non tabellari, come export testuali Web of Science, Cochrane, PubMed MEDLINE e record XML PubMed. Questi file non possono essere trattati sempre con una semplice lettura CSV/Excel, perche' contengono record separati da tag, righe di continuazione e campi ripetuti. I parser trasformano quindi il contenuto grezzo in liste

di dizionari Python, senza applicare ancora lo schema finale: la standardizzazione vera e propria resta compito di `convert2df()`.

La traccia richiede anche un punto di ingresso simile a `convert2df()` di bibliometrix in R. Questo viene risolto con la funzione `convert2df()` in www/services/standardizer.py. Tutti i record, indipendentemente dalla sorgente, passano da questa funzione prima di essere caricati nella dashboard. Questo evita che ogni funzione analitica debba conoscere il formato originale di Scopus, PubMed, Dimensions, Lens, Cochrane, OpenAlex o Web of Science.

Un altro requisito e' l'uso di dizionari di mapping invece di logica hardcoded. Questo viene gestito con `FORMAT_FUNCTIONS_MAP`, che collega ogni campo target dello schema Bibliometrix alla funzione che lo produce, e con `SOURCE_NAME_MAP`, che traduce i nomi normalizzati delle sorgenti negli identificativi attesi dalle funzioni legacy.

La traccia richiede di standardizzare i dati nello schema interno simile a Web of Science. Questo viene risolto producendo sempre un DataFrame con le colonne target richieste, tra cui `DB`, `UT`, `DI`, `PMID`, `TI`, `SO`, `PY`, `TC`, `AU`, `AF`, `C1`, `CR`, `DE`, `ID` e `SR`. Se una sorgente non fornisce uno specifico campo, la colonna viene comunque creata con un valore di default coerente, invece di essere lasciata assente.

Per quanto riguarda i tipi, la traccia richiede contratti forti e gestione esplicita dei valori mancanti. Questo viene risolto con `COLUMN_TYPE_CONTRACTS`, che definisce il tipo atteso per ogni colonna. I campi testuali vengono convertiti in stringhe, `TC` viene convertito in intero, mentre campi multi-valore come `AU`, `AF`, `C1`, `CR`, `DE` e `ID` vengono convertiti in liste Python. I valori `NaN` o `None` non vengono lasciati nel DataFrame finale: per i campi testuali diventano stringhe vuote, per i numerici diventano `0`, per i campi multi-valore diventano liste vuote.

La traccia specifica che i campi multi-valore devono essere gestiti come liste e, se esportati in CSV, serializzati con delimitatore interno. Questo viene risolto mantenendo internamente questi campi come `list`, così le funzioni analitiche possono usarli correttamente. Quando serve esportare in formato piatto, la funzione `serialize_for_csv()` converte le liste usando il delimitatore `;`.

La traccia richiede anche la generazione del campo calcolato `SR`, fondamentale per le analisi citazionali. `SR` e' una breve etichetta del documento, costruita di solito con primo autore, anno e fonte, ad esempio "Rossi, 2024, Journal Name". Per le sorgenti gestite dai formatters legacy, il campo viene prodotto riutilizzando la funzione già presente in `format_functions.py`, cioè `format_sr_column`. Se questa funzione non riesce a produrre `SR`, per esempio perché la sorgente non ha lo stesso formato di Web of Science, lo standardizer prova a costruirlo usando i campi già standardizzati disponibili: primo autore da `AU`, anno da `PY` e fonte da `SO`. Questo non crea informazioni nuove, ma usa i metadati già presenti per ottenere un identificatore documento quando possibile.

La validazione richiesta dalla traccia viene risolta in `validation.py`. Prima di considerare concluso il processo ETL, `validate_record_contract()` controlla i singoli record e `validate_dataframe_contract()` controlla il DataFrame finale. Questi controlli verificano presenza delle colonne, assenza di valori nulli non gestiti e rispetto dei tipi Python attesi.

La traccia richiede infine di dimostrare che il DataFrame standardizzato funzioni con 5-10 funzioni analitiche del progetto. Per questo sono state selezionate sette funzioni disponibili dalla dashboard Biblioshiny: produzione scientifica annuale, citazioni medie per anno, fonti più rilevanti, legge di Bradford, autori più rilevanti, legge di Lotka e documenti più citati globalmente. Dove le funzioni continuavano a fallire per

assunzioni troppo rigide su Web of Science o su oggetti reattivi Shiny, sono state applicate patch minime e documentate.

Per il livello avanzato, la traccia richiede il recupero dati via API da piattaforme open-access come PubMed o OpenAlex. Questo viene gestito in `api_retriever.py`, che recupera i record tramite query testuale, gestisce paginazione, retry e limiti di richiesta, e poi passa i record alla stessa funzione `convert2df()`. La parte importante e' che il flusso API non introduce una seconda pipeline parallela: usa lo stesso standardizer dei file manuali.

Architettura ETL

La soluzione e' organizzata in moduli con responsabilita' separate. `file_extractor.py` coordina la lettura dei file manuali; quando il formato non e' tabellare, delega a `parsers.py`, che interpreta export testuali o XML e li converte in record grezzi strutturati. `api_retriever.py` copre il flusso avanzato da OpenAlex e PubMed, trasformando le risposte API in record intermedi compatibili con la stessa pipeline. `standardizer.py` contiene `convert2df()`, il dispatcher, i mapping e i contratti di tipo. `validation.py` verifica che record e DataFrame finali rispettino lo schema atteso.

Questa separazione evita di duplicare logica nei parser, nella dashboard e nelle funzioni analitiche: le sorgenti entrano nella pipeline in modi diversi, ma prima dell'analisi passano tutte dallo stesso standardizer.

Integrazione nella dashboard

La dashboard Shiny carica ora i dati passando dalla pipeline ETL.

Per i file grezzi il flusso e': `extract_from_file()` legge i record, `convert2df(..., validate=True)` li standardizza e valida, `prepare_dataframe_for_app()` li prepara per la dashboard e il risultato viene salvato nel DataFrame reattivo di Shiny.

Per file Excel/CSV gia' standardizzati, i record vengono comunque ricaricati tramite `convert2df()`. Questo e' necessario perche' un file esportato puo' serializzare le liste come stringhe, mentre la dashboard e le funzioni analitiche si aspettano oggetti Python di tipo `list`.

Per l'importazione via API il flusso e' analogo: `extract_data()` recupera i record, `convert2df(..., file_type="api", validate=True)` li converte nello schema standardizzato, `prepare_dataframe_for_app()` li prepara per l'applicazione e il risultato viene caricato nel DataFrame reattivo di Shiny.

In questo modo file manuali e dati recuperati via API usano la stessa logica di trasformazione e validazione.

Funzioni analitiche testate e patchate

La traccia richiede di testare il DataFrame standardizzato su 5-10 funzioni analitiche. Sono state selezionate funzioni disponibili direttamente dalla dashboard Biblioshiny e rappresentative di diverse parti dello schema.

1. Produzione scientifica annuale

La funzione coinvolta e' `get_annual_production()`, utilizzabile dalla dashboard nel percorso `Overview -> Annual Scientific Production`.

Problema trovato:

La funzione originale assumeva che **PY** fosse già valido e direttamente utilizzabile. Con dati provenienti da sorgenti diverse, gli anni possono arrivare come stringhe o valori non validi.

Patch applicata:

- validazione della colonna **PY**;
- conversione numerica degli anni;
- rimozione degli anni non validi;
- completamento degli anni mancanti con frequenza **0**;
- compatibilità sia con valori reattivi Shiny sia con DataFrame pandas diretti.

Motivo:

La standardizzazione produce una colonna **PY**, ma la funzione deve comunque essere robusta rispetto a formati diversi provenienti dalle sorgenti originali.

2. Citazioni medie per anno

La funzione coinvolta è `get_average_citations()`, utilizzabile dalla dashboard nel percorso **Overview -> Average Citations per Year**.

Problema trovato:

La funzione originale assumeva che **PY** e **TC** fossero già numerici. Alcune sorgenti, in particolare PubMed, possono non fornire citazioni globali.

Patch applicata:

- validazione delle colonne **PY** e **TC**;
- conversione numerica di **PY** e **TC**;
- trattamento delle citazioni mancanti come **0**;
- compatibilità con dashboard e DataFrame pandas diretti.

Motivo:

Le funzioni basate sulle citazioni non devono fallire quando una sorgente non esporta metadati citazionali.

3. Fonti più rilevanti

La funzione coinvolta è `get_relevant_sources()`, utilizzabile dalla dashboard nel percorso **Sources -> Most Relevant Sources**.

Problema trovato:

La funzione originale assumeva che **S0** fosse sempre presente e significativo. Valori vuoti potevano essere conteggiati come fonti valide.

Patch applicata:

- validazione della presenza di **S0**;
- rimozione di valori mancanti o vuoti;
- gestione più sicura delle slice del DataFrame prima dell'aggiunta di colonne temporanee.

Motivo:

La funzione misura la frequenza delle riviste/fonti. Conteggiare valori vuoti avrebbe prodotto risultati fuorvianti.

4. Legge di Bradford

La funzione coinvolta e' `get_bradford_law()`, utilizzabile dalla dashboard nel percorso [Sources -> Bradford's Law](#).

Problema trovato:

La funzione originale assumeva che le fonti fossero sempre presenti e che il dataset fosse abbastanza grande per calcolare e visualizzare correttamente le zone.

Patch applicata:

- validazione di `SO`;
- rimozione delle fonti vuote;
- gestione robusta degli indici del grafico per dataset piccoli.

Motivo:

La legge di Bradford dipende interamente dalla frequenza delle fonti. Dataset piccoli o incompleti non devono causare errori di indice.

5. Autori piu' rilevanti

La funzione coinvolta e' `get_relevant_authors()`, utilizzabile dalla dashboard nel percorso [Authors -> Most Relevant Authors](#).

Problema trovato:

La funzione originale assumeva che `AU` contenesse sempre liste valide di autori. Inoltre mescolava valori tecnici della dashboard, come `percentage` e `freq_measure`, con le etichette della tabella.

Patch applicata:

- validazione della colonna `AU`;
- accettazione solo di valori lista;
- rimozione di liste autore vuote;
- normalizzazione delle opzioni di frequenza della dashboard;
- correzione del conteggio frazionato per evitare problemi con indici non consecutivi dopo i filtri.

Motivo:

`AU` e' un campo multi-valore e deve essere una lista dopo la standardizzazione. Se venissero usate stringhe o valori non validi, i conteggi degli autori potrebbero essere errati.

6. Legge di Lotka

La funzione coinvolta e' `get_lotka_law()`, utilizzabile dalla dashboard nel percorso [Authors -> Lotka's Law](#).

Problema trovato:

La funzione originale assumeva autori validi in formato lista e almeno due livelli di produttività per stimare la curva teorica.

Patch applicata:

- validazione di **AU**;
- filtro dei valori autore non validi;
- gestione del caso con un solo livello di produttività senza chiamare **np.polyfit()**.

Motivo:

np.polyfit() richiede almeno due punti per stimare una curva. Dataset piccoli o uniformi devono comunque produrre un output valido.

7. Documenti più citati globalmente

La funzione coinvolta è **get_cited_documents()**, utilizzabile dalla dashboard nel percorso **Documents -> Most Global Cited Documents**.

Problema trovato:

La funzione dipende da **SR**, **DI**, **TC** e **PY**. Poteva fallire quando le citazioni erano mancanti o quando la media annuale delle citazioni era pari a zero.

Patch applicata:

- validazione delle colonne richieste;
- conversione numerica di **PY** e **TC**;
- riuso di **SR** prodotto dalla standardizzazione;
- validazione della metrica di ranking selezionata;
- gestione degli anni con media citazionale pari a zero nel calcolo di **NormalizedTC**;
- gestione robusta della dimensione dei marker quando tutte le citazioni sono zero.

Motivo:

Questa funzione permette di verificare se identificatori dei documenti e campi citazionali sono stati standardizzati correttamente. Deve inoltre funzionare anche con sorgenti prive di citazioni.

Evidenze di esecuzione

La pipeline è stata verificata tramite la dashboard Biblioshiny usando file bibliografici presenti nel progetto.

Il flusso di validazione è stato il seguente: caricamento di un export bibliografico grezzo dalla dashboard, selezione della sorgente corrispondente, esecuzione della pipeline ETL, verifica del DataFrame standardizzato caricato nella dashboard, esecuzione delle funzioni analitiche e controllo della produzione di output tabellari e grafici senza crash.

Nota sulle citazioni PubMed

I file PubMed spesso non includono citazioni globali. Di conseguenza, quando si importano dati PubMed, le funzioni basate sulle citazioni possono mostrare citazioni pari a zero o informazioni citazionali limitate.

Questo non e' necessariamente un errore della pipeline ETL. La pipeline garantisce che la colonna **TC** esista e sia numerica, ma non puo' creare informazioni citazionali non presenti nella sorgente originale.

Per questo motivo, PubMed e' piu' adatto a verificare la standardizzazione di metadati, autori, anni, fonti e identificatori; le analisi citazionali sono invece piu' informative con sorgenti che esportano citazioni, come Scopus, Web of Science, Dimensions o OpenAlex.

Flusso avanzato via API

Il progetto include anche un flusso avanzato per OpenAlex e PubMed. La parte rilevante, rispetto alla traccia, e' che i record recuperati via API non seguono una pipeline separata: dopo il recupero con `extract_data()`, vengono passati a `convert2df()`, validati e caricati nello stesso DataFrame usato dalla dashboard. In questo modo file manuali e dati ottenuti automaticamente condividono la stessa logica di standardizzazione. VEDERE SE STA SCRITTO PRIMA

File principali modificati

I principali file ETL coinvolti sono `www/services/file_extractor.py`, `www/services/parsers.py`, `www/services/api_retriever.py`, `www/services/standardizer.py`, `www/services/validation.py` e `www/services/format_functions.py`.

Le funzioni analitiche patchate sono `get_annualproduction.py`, `get_averagecitations.py`, `get_relevantsources.py`, `get_bradfordlaw.py`, `get_relevantauthors.py`, `get_lotkalaw.py` e `get_citeddocuments.py`.

Motivazione progettuale

La soluzione mantiene il piu' possibile invariato ciò che è gia' presente in Bibliometrix-Python.

Invece di riscrivere tutte le funzioni, la pipeline ETL standardizza dati eterogenei nello schema gia' atteso dalla dashboard. Sono state patchate solo le funzioni che continuavano a fallire a causa di assunzioni troppo rigide.

Conclusione

Il nostro contributo introduce:

- un punto di ingresso centralizzato simile a `convert2df()`;
- estrazione source-aware da file grezzi;
- recupero dati via API da OpenAlex e PubMed;
- dizionari di mapping per la conversione dello schema;
- contratti di tipo espliciti;
- gestione dei valori mancanti;
- validazione di record e DataFrame finali;
- integrazione nella dashboard Shiny;
- patch di compatibilita' per funzioni analitiche selezionate;

Il risultato e' un DataFrame standardizzato, utilizzabile dalla dashboard Bibliometrix-Python e dalle funzioni analitiche esistenti anche con dati provenienti da piu' sorgenti bibliografiche

COME TESTARLO:

Digitare da terminale: `python -m shiny run app.py`

Per una migliore visualizzazione, consigliamo di aprire l'applicazione mediante l'uso di un browser, da cui è possibile testare il corretto svolgimento della pipeline e delle funzioni analitiche.

LOG DI ESECUZIONE STEP-BY-STEP

Durante questa esecuzione di esempio, sono stati testati sia il Base Level, tramite importazione da parte di una delle sorgenti, che l'Advanced Level, tramite chiamata API.

2026-06-01 19:13:00 | INFO | bibliometrix.app | Avvio applicazione Bibliometrix Shiny 2026-06-01 19:13:39 | INFO | bibliometrix.app | Avvio applicazione Bibliometrix Shiny python : INFO: Started server process [20280]
In riga:1 car:1

- `python -u -W ignore -m shiny run app.py > log_esecuzione.txt 2>&1`

- ```
+ CategoryInfo : NotSpecified: (INFO: Started server process
[20280]:String) [], RemoteException
+ FullyQualifiedErrorId : NativeCommandError
```

INFO: Waiting for application startup. INFO: Application startup complete. INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit) INFO: 127.0.0.1:61087 - "GET / HTTP/1.1" 200 OK INFO: 127.0.0.1:64146 - "WebSocket /websocket/" [accepted] INFO: connection open 2026-06-01 19:13:59 | INFO | bibliometrix.app | Avvio applicazione Bibliometrix Shiny C:\Users\pisac\OneDrive\Desktop\HARDWARE AND SOFTWARE FOR BIG DATA MOD. B\Progetto HwSwB\bibliometrix-python\app.py:8689:  
ShinyDeprecationWarning: `shiny.ui.update_navs()` has been superseded by `shiny.ui.update_navset()` and will be removed in the near future. ui.update\_navs("hidden\_tabs", selected="import") INFO: 127.0.0.1:63745 - "POST /session/a8bd2bfe598b47dad1624b3a1d3581cfc9f1ebbf4a3b0764d602cfd0a243e31/upload/c6b5c80a7d260d236c37a5e2?w= HTTP/1.1" 200 OK 2026-06-01 19:14:24 | INFO | bibliometrix.app | Avvio caricamento dati da interfaccia: azione=1A, database=wos 2026-06-01 19:14:24 | INFO | bibliometrix.app | Import raw data: 1 file ricevuti 2026-06-01 19:14:24 | INFO | bibliometrix.app | Elaborazione file: nome=WoS\_collection.txt, estensione=.txt, sorgente=WEB\_OF\_SCIENCE 2026-06-01 19:14:24 | INFO | www.services.file\_extractor | [WEB\_OF\_SCIENCE] Avvio lettura file: C:\Users\pisac\AppData\Local\Temp\fileupload-k10ziq6d\tmph5h0uke3\0.txt 2026-06-01 19:14:24 | INFO | www.services.file\_extractor | [WEB\_OF\_SCIENCE] Lettura file testuale .txt 2026-06-01 19:14:24 | INFO | www.services.parsers | Parsing Web of Science: C:\Users\pisac\AppData\Local\Temp\fileupload-k10ziq6d\tmph5h0uke3\0.txt 2026-06-01 19:14:24 | INFO | www.services.parsers | Parsing Web of Science completato: 153 record 2026-06-01 19:14:24 | INFO | www.services.file\_extractor | [WEB\_OF\_SCIENCE] Record estratti da file testuale: 153 2026-06-01 19:14:24 | INFO | bibliometrix.app | Estrazione completata per WoS\_collection.txt: record\_grezzi=153 2026-06-01 19:14:24 | INFO | www.services.standardizer | Avvio standardizzazione: source=WEB\_OF\_SCIENCE,

file\_type=.txt, record\_input=153, validate=True, csv\_export=False 2026-06-01 19:14:24 | INFO |  
www.services.standardizer | Record trasformati: validi=153, saltati=0, gia\_standardizzati=0, openalex=0,  
formatter\_legacy=153 2026-06-01 19:14:24 | INFO | www.services.standardizer | Validazione DataFrame  
completata 2026-06-01 19:14:24 | INFO | www.services.standardizer | DataFrame standardizzato pronto:  
righe=153, colonne=24 2026-06-01 19:14:24 | INFO | bibliometrix.app | Standardizzazione completata per  
WoS\_collection.txt: righe=153, colonne=24 2026-06-01 19:14:24 | INFO | bibliometrix.app | Merge DataFrame  
standardizzati completato: righe=153, colonne=24 2026-06-01 19:14:24 | INFO | bibliometrix.app |  
Preparazione per dashboard completata: righe\_valide=153 2026-06-01 19:14:24 | INFO | bibliometrix.app |  
Dataset caricato nello stato reattivo Shiny C:\Users\pisac\OneDrive\Desktop\HARDWARE AND SOFTWARE  
FOR BIG DATA MOD. B\Progetto HwSwB\bibliometrix-python\app.py:8704: ShinyDeprecationWarning:  
`shiny.ui.update_navs()` has been superseded by `shiny.ui.update_navset()` and will be removed in the  
near future. `ui.update_navs("hidden_tabs", selected="API")` 2026-06-01 19:14:44 | INFO | bibliometrix.app |  
Avvio pipeline API da interfaccia: source=openalex, query=machine learning 2026-06-01 19:14:44 | INFO |  
www.services.api\_retriever | Dispatch API: source=openalex, query=machine learning, output\_dir=data\raw  
2026-06-01 19:14:44 | INFO | www.services.api\_retriever | Avvio estrazione OpenAlex: query=machine learning,  
max\_results=100

=====

=====

## ESTRAZIONE OPENALEX - query: 'machine learning'

---

--- Pagina 1 --- 2026-06-01 19:14:44 | INFO | www.services.api\_retriever | Richiesta HTTP: tentativo=1/3,  
url=https://api.openalex.org/works?search=machine+learning&cursor=&per-page=25&mailto=aniello... ->  
GET https://api.openalex.org/works?search=machine+learning&cursor=&per-page=25&mailto=aniello...  
(tentativo 1/3) 2026-06-01 19:14:46 | INFO | www.services.api\_retriever | Richiesta HTTP completata con  
successo: status=200 2026-06-01 19:14:46 | INFO | www.services.api\_retriever | OpenAlex pagina 1: estratti=25,  
totale\_accumulato=25, disponibili=3491621 -> Estratti 25 works (totale: 25 / 3491621 disponibili)

--- Pagina 2 --- 2026-06-01 19:14:46 | INFO | www.services.api\_retriever | Richiesta HTTP: tentativo=1/3,  
url=https://api.openalex.org/works?search=machine+learning&cursor=lls0NDcwLjQ5NiwgMTQ0NzYzMjAw...  
-> GET https://api.openalex.org/works?  
search=machine+learning&cursor=lls0NDcwLjQ5NiwgMTQ0NzYzMjAw... (tentativo 1/3) 2026-06-01 19:14:48 |  
INFO | www.services.api\_retriever | Richiesta HTTP completata con successo: status=200 2026-06-01 19:14:48 |  
INFO | www.services.api\_retriever | OpenAlex pagina 2: estratti=25, totale\_accumulato=50,  
disponibili=3491621 -> Estratti 25 works (totale: 50 / 3491621 disponibili)

--- Pagina 3 --- 2026-06-01 19:14:49 | INFO | www.services.api\_retriever | Richiesta HTTP: tentativo=1/3,  
url=https://api.openalex.org/works?search=machine+learning&cursor=llszMzM2LjM0OTYsIDgxNTE4NDAw... -  
> GET https://api.openalex.org/works?  
search=machine+learning&cursor=llszMzM2LjM0OTYsIDgxNTE4NDAw... (tentativo 1/3) 2026-06-01 19:14:51 |  
INFO | www.services.api\_retriever | Richiesta HTTP completata con successo: status=200 2026-06-01 19:14:51 |  
INFO | www.services.api\_retriever | OpenAlex pagina 3: estratti=25, totale\_accumulato=75,  
disponibili=3491621 -> Estratti 25 works (totale: 75 / 3491621 disponibili)

--- Pagina 4 --- 2026-06-01 19:14:51 | INFO | www.services.api\_retriever | Richiesta HTTP: tentativo=1/3,  
url=https://api.openalex.org/works?search=machine+learning&cursor=llsyNzUyLjY4MjEsIDZMTkxNTUy... ->

GET https://api.openalex.org/works?search=machine+learning&cursor=llsyNzUyLjY4MjEsIDEzMTkxNTUy...  
(tentativo 1/3) 2026-06-01 19:14:53 | INFO | www.services.api\_retriever | Richiesta HTTP completata con  
successo: status=200 2026-06-01 19:14:53 | INFO | www.services.api\_retriever | OpenAlex pagina 4: estratti=25,  
totale\_accumulato=100, disponibili=3491621 -> Estratti 25 works (totale: 100 / 3491621 disponibili) ->  
Raggiunto limite o fine risultati. Dataset completato. 2026-06-01 19:14:53 | INFO | www.services.api\_retriever |  
Salvataggio OpenAlex JSON: file=data\raw\openalex\_20260601\_191453.json, record=100

[SALVATAGGIO] OpenAlex JSON -> data\raw\openalex\_20260601\_191453.json (100 record) 2026-06-01  
19:14:53 | INFO | www.services.api\_retriever | Estrazione OpenAlex completata: record=100 2026-06-01  
19:14:53 | INFO | bibliometrix.app | Estrazione API completata: source=openalex, record\_grezzi=100 2026-06-  
01 19:14:53 | INFO | www.services.standardizer | Avvio standardizzazione: source=OPENALEX, file\_type=api,  
record\_input=100, validate=True, csv\_export=False 2026-06-01 19:14:54 | INFO | www.services.standardizer |  
Record trasformati: validi=100, saltati=0, gia\_standardizzati=0, openalex=100, formatter\_legacy=0 2026-06-01  
19:14:54 | INFO | www.services.standardizer | Validazione DataFrame completata 2026-06-01 19:14:54 | INFO |  
www.services.standardizer | DataFrame standardizzato pronto: righe=100, colonne=24 2026-06-01 19:14:54 |  
INFO | bibliometrix.app | Standardizzazione API completata: source=OPENALEX, righe=100, colonne=24 2026-  
06-01 19:14:54 | INFO | bibliometrix.app | Preparazione dashboard per dati API completata: righe\_valide=100  
2026-06-01 19:14:54 | INFO | bibliometrix.app | Dataset API caricato nello stato reattivo Shiny  
C:\Users\pisac\OneDrive\Desktop\HARDWARE AND SOFTWARE FOR BIG DATA MOD. B\Progetto  
HwSwB\bibliometrix-python\app.py:1152: ShinyDeprecationWarning: shiny.ui.update\_navs() has been  
superseded by shiny.ui.update\_navset() and will be removed in the near future.  
ui.update\_navs("hidden\_tabs", selected="import")